11. Testing and Evaluation Plan

11.1 Purpose of Testing

Testing will be used to check whether the Local Service Problem Diagnosis and Matching System works according to the requirements described in this document. The main goal is to make sure that the system accepts problem descriptions, identifies suitable service categories, matches requests with service providers, and stores request details correctly.

I will test the system at different stages of development. Individual functions will be tested first, followed by tests of the connections between the classification model, Flask application, and SQLite database. After these tests, I will test the complete system from the user's point of view.

The results will be recorded using test inputs, expected results, actual results, screenshots, database records, and performance measurements. Any failed test will be reviewed, corrected, and repeated. The system will be considered ready for its intended use only after the important requirements have passed their tests.

11.2 Testing Methods

The following testing methods will be used during development.

Testing method	Purpose	Evidence to collect
Unit testing	Check individual functions, such as text preprocessing, classification, and input validation.	Test results showing passed and failed cases.
Integration testing	Check whether Flask, the classification model, and SQLite work together correctly.	Screenshots, application output, and database records.
System testing	Test the complete application using normal and incorrect inputs.	Completed test cases and observed results.
API testing	Check request and response behavior for any API endpoints included in the final system.	Request data, response status codes, and response contents.
Usability testing	Check whether users can submit a problem and understand the result without difficulty.	Feedback from test users and a record of issues found.
Performance testing	Measure response times when the application processes requests.	Response-time measurements under a defined workload.
Security testing	Check input validation, database access, authentication, and protection of stored information.	Security test results and evidence of rejected unauthorized requests.
Acceptance testing	Check whether the completed system meets the main project requirements.	A requirement checklist with pass or fail results.

Some tests, such as API, authentication, and security testing, will be completed when the corresponding features have been implemented.

11.3 Functional Testing

Functional testing will check whether each major feature behaves as expected. The following table defines the planned test cases.

Test ID	Requirement	Test procedure	Expected result
FT-01	The system shall accept a problem description.	Enter a valid description of a household problem and submit it.	The application accepts the description and processes the request.
FT-02	The system shall classify a problem into a service category.	Enter “The kitchen sink pipe is leaking.”	The model returns a service category, expected to be Plumbing for this example.
FT-03	The system shall reject an empty problem description.	Submit the form without entering any text.	The application displays a validation message and does not save an invalid request.
FT-04	The system shall save submitted requests.	Submit a valid description and inspect the SQLite database.	The database contains the submitted description, predicted category, and associated request ID.
FT-05	The system shall match a request with suitable service providers.	Submit a problem for a category with registered providers.	The system displays providers whose service categories match the request.
FT-06	The system shall allow users to view request history.	Submit multiple requests and open the history page.	The correct saved requests are displayed with their available details.
FT-07	The system shall allow request status updates.	Change a request status through an authorized provider action.	The new status is saved and displayed correctly.
FT-08	The system shall handle an unrecognized problem.	Submit a description that does not clearly fit the available categories.	The system avoids presenting an unsupported category as a reliable diagnosis and provides an appropriate message or review option.

The classification results will be checked against the expected categories in the test dataset. Because the current model uses a small illustrative dataset, a correct prediction in one example will not be treated as proof that the model is accurate for all household problems.

11.4 Integration Testing

Integration testing will determine whether the main technical components communicate and exchange information correctly.

Test ID	Components	Test procedure	Evidence of success

IT-01	HTML form and Flask	Submit a problem through the browser.	Flask receives the submitted description without an application error.
IT-02	Flask and classification model	Submit several descriptions from the available service categories.	The model returns a category for each valid input.
IT-03	Flask and SQLite	Submit a request and inspect the database.	The saved record matches the submitted description and prediction.
IT-04	Classification model and database	Submit a request and compare the displayed prediction with the stored category.	The displayed and stored categories are the same.
IT-05	Request processing and provider matching	Submit a request for a category with registered providers.	The matching component returns eligible providers from the database.

The current proof of concept has already demonstrated a basic classification and request-saving workflow. The remaining integration tests will be performed as additional components are implemented. Database persistence should be verified directly by inspecting the stored record rather than relying only on the message displayed in the browser.

11.5 Evaluation of the Classification Model

The classification model is an important part of the project because the predicted category affects which service providers may be recommended. I will evaluate the model using a larger labeled dataset containing examples from Plumbing, Electrical, and Appliance Repair.

The dataset will be divided into training and testing sets. The test data will not be used to train the model. I will compare the predicted categories with the correct labels and calculate the following measures:

- Accuracy: the proportion of all test examples classified correctly.
- Precision: how often predictions for a category are correct.
- Recall: how many actual examples of a category are identified correctly.
- F1-score: a combined measure of precision and recall.

Evaluation ID	Test activity	Evidence to collect	Planned success criterion
ML-01	Evaluate predictions on held-out test data.	Accuracy score and classification report.	Target at least 80% overall accuracy on the held-out dataset.
ML-02	Check performance for each service category.	Precision, recall, and F1-score for each category.	Target an F1-score of at least 0.75 for each category.
ML-03	Test descriptions with different wording.	Predictions for representative alternative descriptions.	Record incorrect or unexpected predictions and investigate their causes.

ML-04	Test descriptions that do not fit the trained categories.	Model output and system response.	The application should provide a safe fallback rather than treating an unsupported prediction as a confirmed diagnosis.

These values are proposed evaluation targets, not measured results. If the model does not meet them, I will review the training data, preprocessing method, and classification approach before deciding whether it is suitable for the final system.

11.6 Nonfunctional Testing

Nonfunctional testing will check performance, usability, reliability, and security.

Test ID	Quality requirement	Test procedure	Evidence of success
NFT-01	Response time	Submit 100 representative requests under normal local operating conditions and measure the time from submission to completed response.	At least 95 of the 100 requests complete within two seconds.
NFT-02	Data reliability	Submit a request, stop and restart the application, and inspect the database.	The saved request remains available after restarting the application.
NFT-03	Input validation	Submit blank text, very long descriptions, and unexpected characters.	Invalid inputs are handled safely without an unhandled application error.
NFT-04	Usability	Ask at least five test users to submit a problem and interpret the result.	At least four of the five users complete the task without assistance and can identify the predicted category.
NFT-05	Database security	Submit text containing SQL injection patterns and inspect the database behavior.	Input is handled safely, database structure remains unchanged, and unrelated records are not exposed or modified.
NFT-06	Access control	Attempt to access restricted functions without the required authorization after authentication is implemented.	Unauthorized access is rejected.
NFT-07	Reliability under repeated use	Submit 100 requests in a controlled test and check the saved records.	All valid requests are processed without data loss or unhandled application errors.

The two-second response-time target is intended for a local or otherwise defined normal test environment. I will record the computer configuration, workload, and test conditions so that the results can be interpreted correctly. If the final deployment environment differs from the development environment, performance will need to be tested again.

11.7 Acceptance Testing

Acceptance testing will be performed after the main system features have been implemented. It will check whether the application meets the functional and nonfunctional requirements from the perspective of its intended users.

The acceptance tests will cover submitting a household problem, receiving a predicted service category, finding a suitable provider, saving the request, viewing its history, and checking request status updates.

Acceptance ID	Requirement to verify	Evidence required	Acceptance condition
AT-01	Problem submission and classification	Test input and displayed prediction.	A valid problem can be submitted and a category is returned.
AT-02	Request storage	Corresponding SQLite database record.	The submitted details and prediction are stored correctly.
AT-03	Provider matching	Matching results and provider category records.	Only eligible providers for the requested category are returned.
AT-04	Request history and status	History page and database records.	The correct request details and current status are displayed.
AT-05	Classification quality	Held-out evaluation results.	The model meets the agreed classification targets or its limitations are documented and the approach is revised.
AT-06	Performance and reliability	Timing measurements and repeated-request results.	The defined response-time and reliability targets are met.

A requirement will be marked as passed only when the relevant test has been performed and supporting evidence has been recorded. If a test fails, I will document the issue, make the necessary changes, and repeat the test. Features that have not yet been implemented will remain marked as pending rather than being reported as successful.

11.8 Recording and Reporting Test Results

I will maintain a test record containing the test ID, date, input, expected result, actual result, status, and evidence location. Screenshots will be included where they help explain the result, while database checks and model evaluation reports will provide additional evidence.

The current proof of concept provides an initial test result: the description “The kitchen sink pipe is leaking” produced the predicted category Plumbing, a model confidence value of 0.51, and request ID 1. I will verify the corresponding database record and conduct additional tests before marking the related requirements as fully tested. The final evaluation will be based on the collected results rather than the appearance of the application alone. This approach will help identify missing features, incorrect predictions, database problems, and performance or security weaknesses before the system is considered complete.